

Trabalho da disciplina de Álgebra Linear – Análise de Componentes Principais (PCA)

Adam Stader Nobre (FATEC RUBENS LARA)

adam.nobre@fatec.sp.gov.br

Tamara Moreira de Tarso Pereira (FATEC RUBENS LARA)

tamara.pereira2@fatec.sp.gov.br

1.Introdução

Na Ciência de Dados, temos como uma das principais habilidades a serem desenvolvidas, a análise de dados. Utilizando vários tipos de ferramentas conseguimos visualizar os dados para então conseguir interpretá-los, seja utilizando ferramentas de estatística, operações matemáticas, visualização, entre outras. Neste trabalho vamos explorar um dos desafios em diversas áreas da estatística: a análise de grandes conjuntos de dados multivariados. A alta dimensionalidade dos dados pode dificultar a visualização, interpretação e a aplicação em modelos como os de aprendizado de máquina. A Análise de Componentes Principais (PCA) é uma técnica estatística que tem como objetivo reduzir a dimensionalidade de dados, transformando um conjunto de variáveis em um conjunto menor de novas variáveis que chamamos de componentes principais, que são combinações lineares das variáveis originais e são escolhidos de forma a capturar a maior parte da variância dos dados.

Para explorar a aplicação do PCA em um conjunto de dados, o qual foi escolhido o “Iris”, um conjunto de dados de flores que descreve três espécies da flor Íris (Iris Setosa, Iris Versicolor e Iris Virginica) com base em quatro características (Comprimento e Largura de Pétala e Sépala). O objetivo é identificar os componentes principais para classificação através do PCA e obter uma representação mais compacta e informativa dos dados, facilitando a análise e interpretação dos resultados.

2.Base de Dados

2.1. Conjunto de dados

Para esse estudo foi escolhida a base de dados Iris Flower Dataset, que foi coletado por Ronald Fisher e descreve três espécies de Íris com base em quatro características. Ele possui 5 colunas por 150 linhas, sendo as colunas: sepal_length (comprimento da sépala), sepal_width (largura da sépala), petal_length (comprimento da pétala), petal_width (largura da pétala) e species (espécie). Já as linhas são divididas exatamente 50 exemplos de cada uma das 3 espécies da flor Íris (Iris Setosa, Iris Versicolor e Iris Virginica).

```
#Carregando o DataSet
iris = pd.read_csv("Iris.csv")
iris.info()

✓ 0.0s

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 150 entries, 0 to 149
Data columns (total 5 columns):
#   Column          Non-Null Count  Dtype
---  -
0   sepal_length    150 non-null   float64
1   sepal_width     150 non-null   float64
2   petal_length    150 non-null   float64
3   petal_width     150 non-null   float64
4   species         150 non-null   object
dtypes: float64(4), object(1)
memory usage: 6.0+ KB
```

2.2. Análise de Componentes Principais

Das 5 variáveis que temos, 1 é categórica (espécie) e as outras 4 são numéricas, portanto temos 4 dimensões para explorar aplicando o PCA e visualizar as relações entre as diferentes espécies de Íris e identificamos após a padronização que temos dois componentes principais, sendo um deles relacionado à forma da pétala e o outro relacionado ao tamanho geral da flor. Utilizando o Python como linguagem e as bibliotecas Pandas, Numpy, Scikit-learn, Matplotlib e Seaborn para conseguir ler a base de dados, transformar os dados, realizar as operações e visualizar os dados.

Utilizando a biblioteca Pandas para ler a base de dados e em seguida carregá-la utilizando a outra biblioteca Seaborn. Após esse processo é necessário preparar os dados, separando as variáveis numéricas e escalonar para obter a matriz com as variáveis padronizadas com o objetivo de obter a matriz de covariância, autovalores e autovetores:

```
# Leitura do dataset pelo pandas

iris = pd.read_csv("Iris.csv")

# Carregando o dataset Iris pelo Seaborn

iris = sns.load_dataset('iris')

# Separando as features numéricas

X = iris.iloc[:, :-1] # Selecionando todas as linhas e todas as colunas exceto a última (espécie)

# Padronizando os Dados pelo Escalonamento

escalar = StandardScaler()
X_escalonado = escalar.fit_transform(X)
```

Agora começamos com o PCA colocando ele em uma variável e calculamos a Matriz de Covariância, Autovalores e Autovetores usando a biblioteca Numpy:

```
# PCA

pca = PCA()
pca.fit(X_escalonado)
```

✓ 0.0s

Python

```
# Obtendo a matriz de covariância dos dados padronizados

Matriz_covariancia = np.cov(X_escalonado.T)

# Calculando os autovalores e autovetores

Autovalores, Autovetores = np.linalg.eig(Matriz_covariancia)

# Ordenando os autovalores e autovetores em ordem decrescente

Indice = Autovalores.argsort()[::-1] #Armazena os autovalores em ordem decrescente
Autovalores = Autovalores[Indice]
Autovetores = Autovetores[:,Indice]

print("Matriz de Covariância: \n\n",Matriz_covariancia)
print("\nAutovalores: \n\n", Autovalores)
print("\nAutovetores:\n\n ", Autovetores)
```

✓ 0.0s

Python

Matriz de Covariância:

```
[[ 1.00671141 -0.11835884  0.87760447  0.82343066]
 [-0.11835884  1.00671141 -0.43131554 -0.36858315]
 [ 0.87760447 -0.43131554  1.00671141  0.96932762]
 [ 0.82343066 -0.36858315  0.96932762  1.00671141]]
```

Autovalores:

```
[2.93808505 0.9201649  0.14774182 0.02085386]
```

Autovetores:

```
[[ 0.52106591 -0.37741762 -0.71956635  0.26128628]
 [-0.26934744 -0.92329566  0.24438178 -0.12350962]
 [ 0.5804131  -0.02449161  0.14212637 -0.80144925]
 [ 0.56485654 -0.06694199  0.63427274  0.52359713]]
```

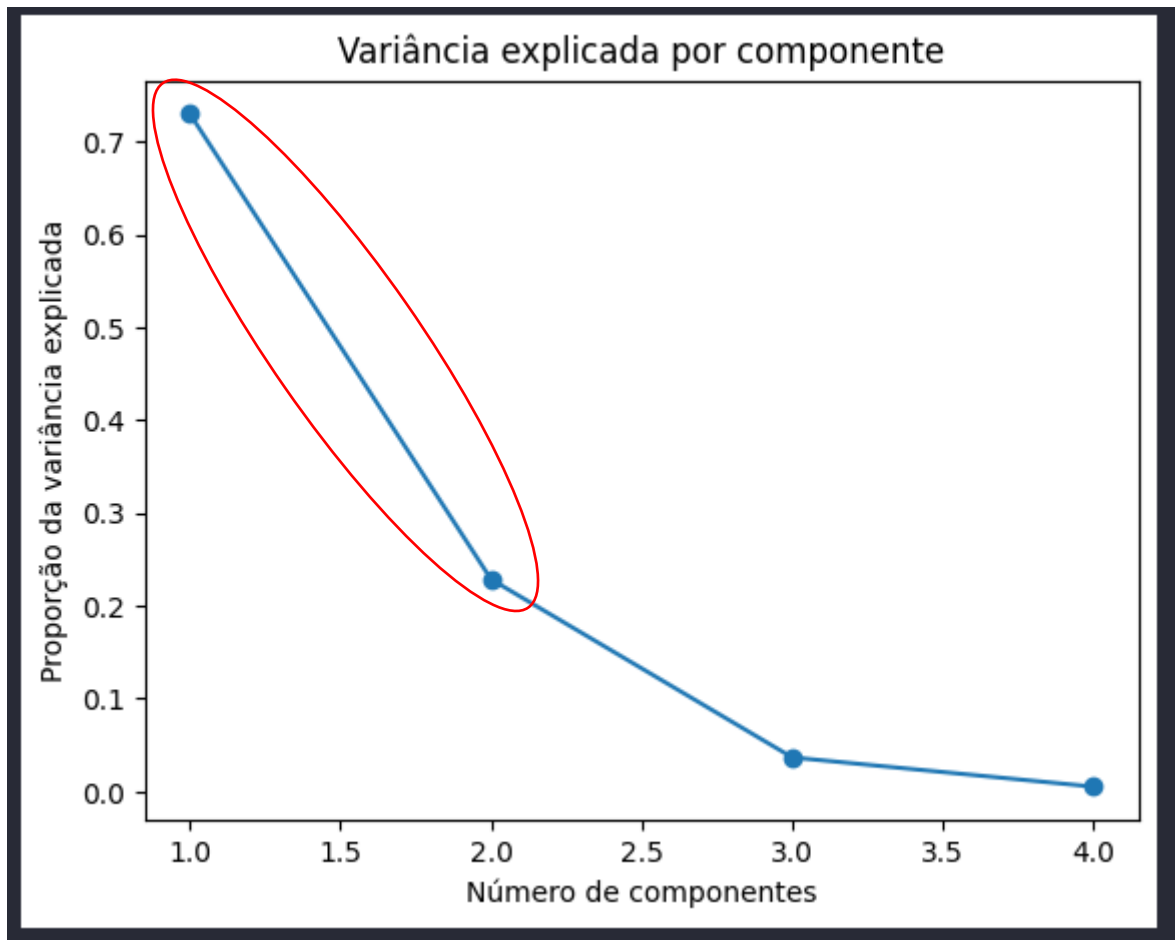
Assim utilizando as bibliotecas Scikit-learn e Matplotlib conseguimos a variância explicada de cada componente permitindo assim utilizar o “Método do Cotovelo” para descobrir quantos componentes tem a maior parte da variância (componentes principais) e assim verificamos que são dois componentes principais:

```
# Obtendo a proporção da variância explicada por cada componente
Matriz_covariancia = pca.explained_variance_ratio_

# Método do "Cotovelo" para descobrir quantos componentes pegam maior parte da variância

plt.plot(range(1, len(Matriz_covariancia)+1), Matriz_covariancia, marker='o')
plt.xlabel('Número de componentes')
plt.ylabel('Proporção da variância explicada')
plt.title('Variância explicada por componente')
plt.show()
```

✓ 0.1s



Com o “Método do Cotovelo”, fica visível pela curva que a maior parte da variância nos dados é explicada pelos primeiros dois componentes principais e a partir do segundo componente, não temos uma queda tão grande na variância. Então agora vamos transformar o PCA de 4 dimensões para 2.

```
pca = PCA(n_components=2)  
ComponentesPrincipais = pca.fit_transform(X_escalonado)
```

✓ 0.0s

E por fim, vamos gerar o Gráfico de Dispersão juntando a variável categórica que tínhamos (Espécie) com a biblioteca Pandas, e teremos nosso resultado utilizando a biblioteca Seaborn junto da Matplotlib:

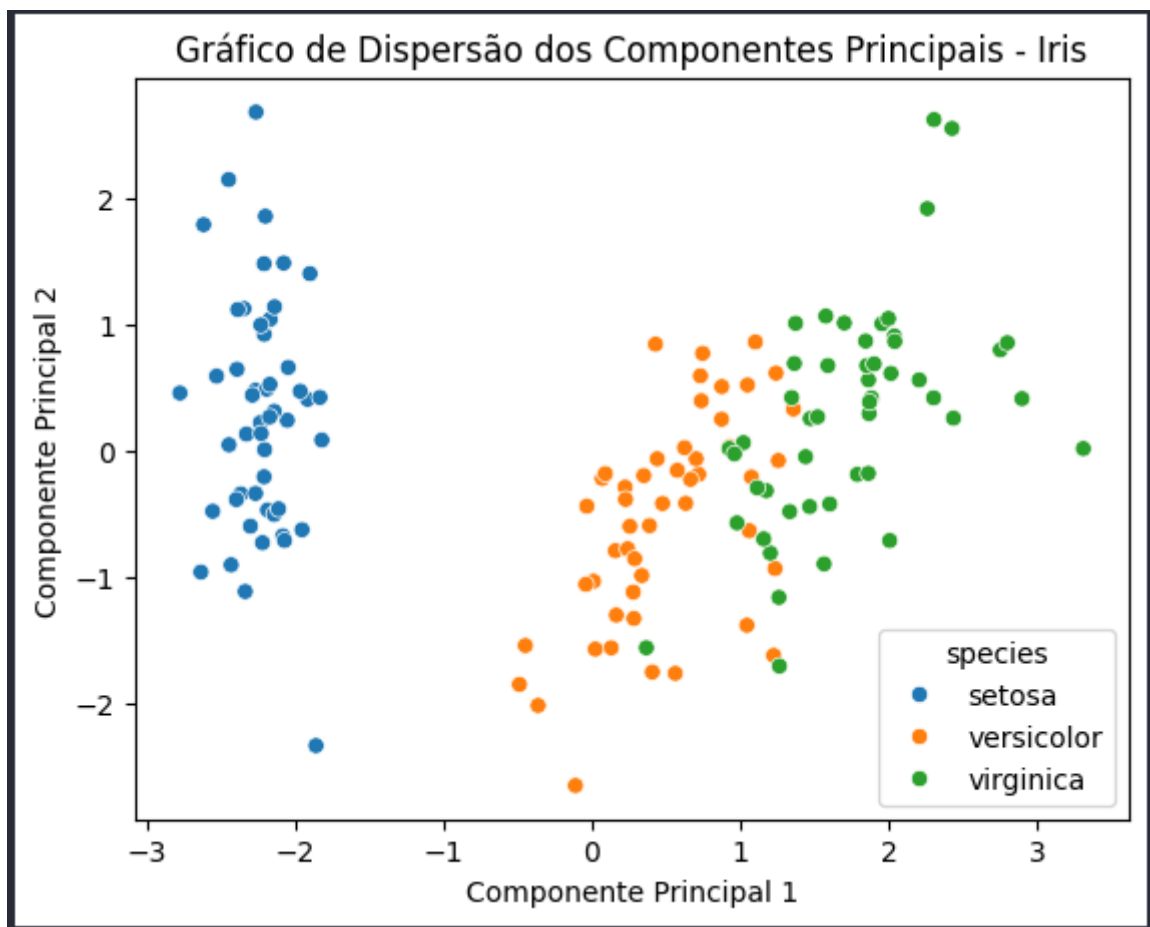
```

df = pd.DataFrame(data = ComponentesPrincipais, columns = ['Componente Principal 1', 'Componente Principal 2'])
finalDf = pd.concat([df, pd.DataFrame(iris['species'], columns = ['species'])], axis = 1)
✓ 0.0s

# Gráfico de dispersão colorido por espécie

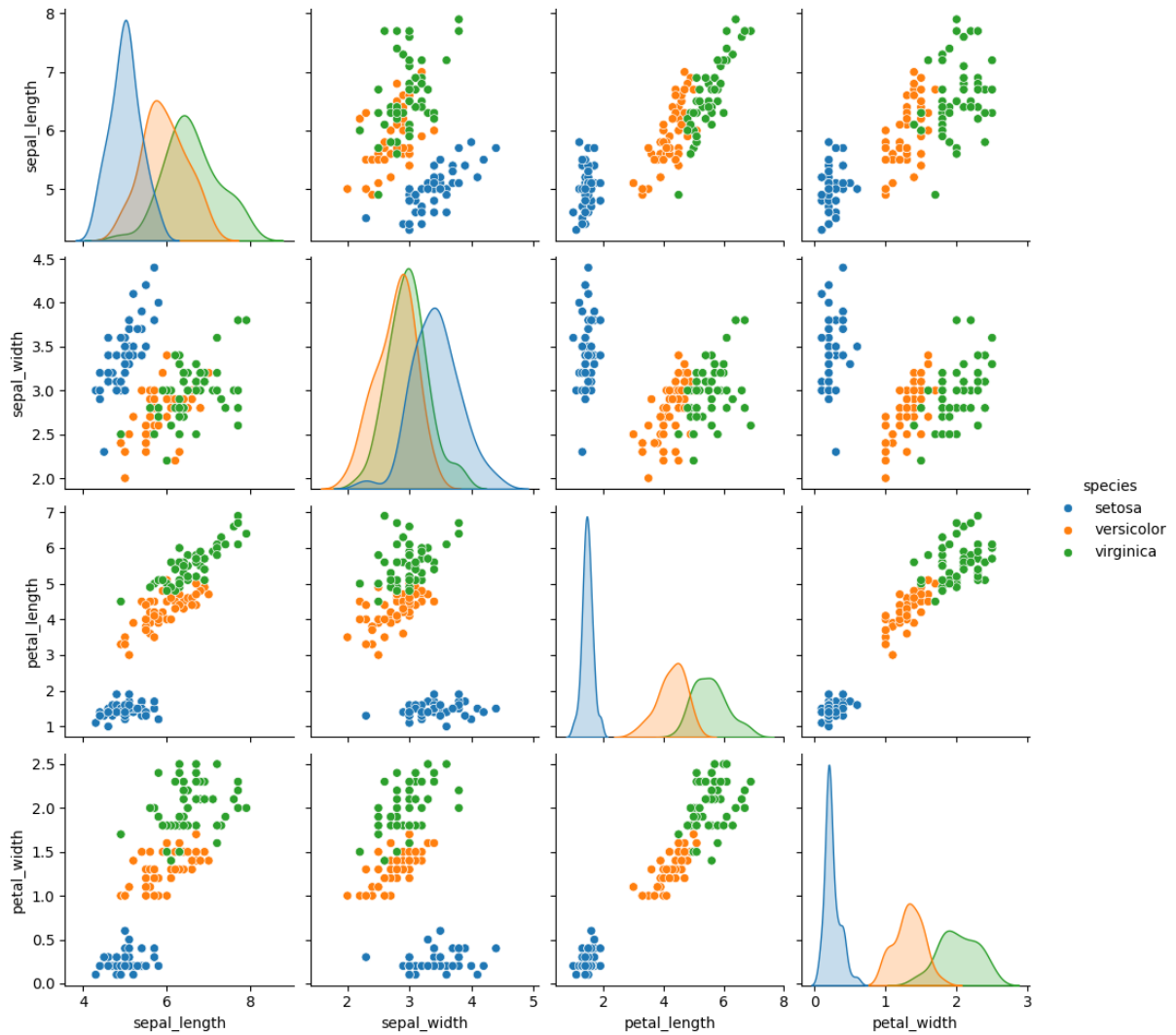
sns.scatterplot(x="Componente Principal 1", y="Componente Principal 2", hue="species", data=finalDf)
plt.title('Gráfico de Dispersão dos Componentes Principais - Iris')
plt.show()
✓ 0.1s

```



Resultados e Considerações Finais

Para fins de comparação, isso é o que teríamos se fossemos analisar as 4 dimensões:



E como o método do PCA reduzimos de 16 gráficos de dispersão para apenas 1 com uma análise bem assertiva, então é uma técnica estatística poderosa e que realmente nos ajuda a analisar os dados de maneira mais prática, ajudando na visualização dos dados, performance dos modelos e extração de características principais.

Referências

GITHUB. **Machine Learning/PCA.ipynb**. Disponível em:

https://github.com/arnaldog12/Machine_Learning/blob/master/PCA.ipynb.

Scikit-Learn. **Principal Component Analysis (PCA)**. Disponível em: https://scikit-learn.org/dev/auto_examples/decomposition/plot_pca_iris.html.

IBM. **O que é a análise de componentes principais (PCA)?**. Disponível em:

<https://www.ibm.com/br-pt/topics/principal-component-analysis>.

The Avila. **Redução de Dimensionalidade com Algoritmo PCA**. Disponível em:

<https://theavila.github.io/2020/07/16/PCA-Iris/>.

YouTube. **Análise de Componentes Principais (PCA) em Python - Machine Learning**

24.2. Disponível em: <https://www.youtube.com/watch?v=4HU3lqj0Cc8>.